Avaliação 2 – Indexação e Processamento de Consultas

Universidade Federal do Rio de Janeiro

Disciplina	Construção de Banco de Dados - 2026.2
Professor	Milton R. Ramirez
Trabalho	Avaliação 2 – Indexação e Processamento de Consultas
Prazo de entrega	22 de novembro de 2026

O trabalho consiste em construir, com auxílio de IA, uma **bancada de teste** para avaliação do desempenho do processamento de operações relacionais. A bancada deve fornecer as medidas que permitam fazer uma análise comparativa entre algumas das possibilidades de configurações do Gerente de Armazenamento para o processamento de cargas de trabalho típicas de sistemas de banco de dados relacionais. As medidas a serem obtidas com a banca de teste se referem, principalmente, ao número de blocos transferidos entre memória secundária e o *buffer pool*.

> Em resumo, a pergunta a ser respondida neste trabalho é: *qual configuração do SBD é mais adequada para realizar o processamento eficiente de uma dada carga de trabalho para uma determinada base de dados?*

1. Arquitetura sugerida

Sugerimos o desenvolvimento da bancada de teste em *quatro* camadas, cada uma servindo-se apenas da interface da camada imediatamente inferior, como descrito a seguir.

1. **Bancada de Teste (orquestrador)**: carrega (ou gera) a base de dados de teste, instancia cada configuração a ser avaliada, aciona a execução da lista de operações a serem testadas, configura e coleta os indicadores, e registra os resultados em formato tabular (CSV, por exemplo).
2. **Mini Monitor de Execução**: biblioteca de primitivas de avaliação, uma por algoritmo de operação relacional, invocadas com o caminho de acesso escolhido explicitamente pelo experimento. Não há processador de SQL, otimizador nem geração de plano de execução (avaliação); os algoritmos são justamente o objeto da comparação para uma dada base de dados.
3. **Gerente de Armazenamento**: oferece as estruturas e controles para a configuração das organizações primárias dos arquivos de dados e métodos de acesso auxiliares. Mantém um mini catálogo (dicionário de dados) com o esquema de cada tabela, sua organização e seus índices e demais parâmetros necessários à execução das operações e sua avaliação de desempenho.
4. **Gerente de Arquivos e Gerente de Buffer Pool**: o primeiro realiza as transferências de blocos entre a memória secundária e o *buffer pool*; o segundo é o mecanismo pelo qual as camadas superiores obtêm as páginas em memória.

Para que as contagens sejam comparáveis entre configurações, recomenda-se que nenhuma camada acima do Gerente de Buffer Pool acesse blocos da memória diretamente.

2. Gerente de Arquivos e Gerente de Buffer Pool

2.1 Gerente de Arquivos

O disco pode ser simulado por arquivos do sistema operacional divididos em blocos de tamanho fixo. Para este módulo, deve-se decidir os seguintes parâmetros:

1. O tamanho dos blocos da memória secundária e a técnica de sua alocação nos arquivos (contígua, encadeada, ou por *extents*)
2. A estrutura do cabeçalho do arquivo (descritor)
3. Como será a gerência de blocos livres
4. Quais contadores de leituras e gravações físicas, por arquivo e globais, distinguindo entre acessos sequenciais e acessos aleatórios (*seeks*). Esse serão usados para gerar as medidas fornecidas pela bancada de testes.

2.2 Gerente de *Buffer Pool*

O grupo deverá definir o tamanho e a organização do *buffer pool*, ou seja:

1. o número total de páginas (e quanto blocos por página)
2. Se a repartição do *buffer pool* repartição será por "tipo" de página ou por operação e quais as áreas (para dados, índices, metadados e catálogo, trabalho reservada às operações de ordenação e junção etc.),
3. Se as áreas são fixas, ou autoajustáveis (dado limite), ou compartilhadas
4. Quais as duas "políticas de substituição" de página a serem comparadas, entre: LRU, MRU, FIFO e Clock (segunda chance)
5. Se haverá ou não o emprego de *toss-immediate*
6. Como se dará a fixação de blocos (*pinned blocks*)
7. Se haverá gravação forçada (*forced output*)
8. Se haverá substituição com prioridade, ou retenção preferencial de páginas como índice e catálogo
9. Se haverá a leitura antecipada (*prefetching*) de blocos

Métricas do buffer a registrar: requisições, acertos, faltas, taxa de acerto, gravações de páginas atualizadas (*dirty page*) na expulsão e gravações forçadas.

3. Organizações primárias dos arquivos de dados

Para este módulo o grupo deve permitir a comparação de três das organizações primárias estudadas: heap, Árvore B+ e Hash, sendo que para os dois últimos terá duas opções de organização primária, uma usando um campo UNIQUE (chave), e outra usando um campo NOT UNIQUE (não chave), ou seja, são cinco tipos de organização primária a serem testadas. Recomenda-se que para cada organização sejam implementadas as operações de arquivo *registro a registro*, como operações de *Open, Reset, Find, FindNext, Read, Insert, Delete, Close* e *Scan*; e de *conjunto a conjunto*, como operações de *FindAll, FindOrdered, Reorganize* (quando pertinente).

Sugerimos a seguir uma lista de decisões que o grupo deve tomar quanto aos parâmetros de configuração para o Gerente de Arquivos:

3.1 Quanto ao formato dos registros e layout dos blocos

1. Tamanho do bloco de registros e fator de blocagem entre 2k, 4k ou 8k
2. Como será a organização dos campos de tamanho fixo e de tamanho variável no registro de dados, tratamento dos campos comuns, campos opcionais, campos multivalorados (*repeating field*), campos estruturados, campos derivados e uso de *null bitmap.*
3. Organização dos registros de dados de tamanho fixo e variável dentro do bloco
4. Como será o tratamento dos registros deletados no bloco e se haverá uso de página com slots (*slotted page*)
5. Se a blocagem será não estendida (*unspanned*) e estendida (*spanned*).

3.2 Para a organização do arquivo de dados em heap files (não ordenado)

1. Como será a gerência da lista de registros deletados: uso de mapa de espaço livres (*free-space map*), uso de lista de registros livres (*free list*)
2. Qual o critério para realizar a compactação dos espaços livres.

3.4 Para a organização do arquivo de dados em Árvore B+

1. Chave de ordenação por campo único ou grupo de campos.
2. Estrutura do nó interno e do nó folha, tamanho do bloco e *fan-out*
3. Taxa de ocupação mínima dos nós para divisão, redistribuição e fusão.
4. Se haverá a carga em massa (*bulk loading*) ou inserção registro a registro.

3.5 Para a organização do arquivo de dados por Hash Externo

1. Usar apenas hashing externo estático, com tamanho da tabela de hash definida para o melhor desempenho segundo a carga de trabalho
2. Definir estrutura e tamanho dos *bucktes*
3. Escolha da função de hashing para a melhor opção (dispersão e baixa colisão) segundo o domínio e a distribuição de valores do campo escolhido para chave de hashing
4. Escolha da política de tratamento de colisão (*bucket overflow*), e tratamento do desbalanceamento (*skew*) e do controle do fator de carga.

4. Métodos de acesso e uso de Índices

A bancada de teste deve permitir as seguintes configurações de métodos de acesso aos arquivos de dos, segundo as suas organizações primárias, e utilizando índices, ou não.

4.2 Métodos de acesso a serem implementados

Organização primária	Método de acesso
Heap	1. Direto para campo sem índice 2. Acesso via índice secundário para campo UNIQUE. Deve ser um Hash 3. Acesso via índice secundário para campo NOT UNIQUE, que deve ser uma árvore B+ sem repetição de chave nas folhas
Árvore B+	1. Direto, via o índice primário, para campo UNIQUE ou NOT UNIQUE

Organização primária	Método de acesso
	2. Acesso via índice secundário *clustering* somente para campo NOT UNIQUE, que deve ser uma árvore B+ sem repetição de chave nas folhas
Hash	1. Direto, via o índice primário para campo UNIQUE (chave primária) 2. Direto, via o índice primário para campo NOT UNIQUE (campo não chave) 3. Acesso via índice secundário *clustering* para campo NOT UNIQUE, que deve ser uma arvore B+ sem repetição de chave nas folhas

5. Operações relacionais a serem testadas

A junção é a operação que melhor discrimina o desempenho de uma configuração, pois exercita simultaneamente a organização primária, os índices disponíveis e a gerência do buffer pool. Deve, por isso, ocupar o centro da avaliação. No tocante as demais operações, e com o intuito de diminuir a carga dos testes, serão testadas apenas a operação de seleção, deixando de lado as operações de projeção, ordenação, conjunto, agrupamento etc.

5.1 Operações de Junção

Cada grupo deverá avaliar a execução dos seguintes três tipos de junções:

1. NATURAL JOIN entre tabelas mestre-detalhe (*master e detail tables*), usando a chave primário (PK) da tabela mestre e a sua correspondente chave estrangeira (FK) na tabela de detalhe,
2. Equijoin entre duas tabelas usando campos UNIQUE que não ditam a organização primária
3. Equijoin entre duas tabelas usando campos NOT UNIQUE que não ditam a organização primária

Cada um deles deverá ser comparado usando os seguintes algoritmos de execução do JOIN:

1. *Nested-loop* em blocos
2. *Sort-merge join*
3. *Partition-hash Join*

Para cada combinação de operação e algoritmos, as seguintes configurações deverão ser testadas:

1. Os campos do predicado da junção
    a. Em ambas as tabelas não possuem método de acesso primário ou secundário (não possuem índices)
    b. Um dos campos é o da organização primária (campo chave da árvore B+, ou a chave do *hashing*) e o da outra tabela não possuem método de acesso primário ou secundário
    c. Em ambas as tabelas são os da respectiva organização primária (campo chave da árvore B+, ou a chave do *hashing*)
    d. Um dos campos é o da organização primária (campo chave da árvore B+, ou a chave do *hashing*) e o da outra tabela possui um índice secundário
    e. Em ambas as tabelas os respectivos campos possuem índices secundários
2. As três combinações de tamanho da relação: menor join menor, menor join maior, maior join maior, sendo que a tabela dita menor é a que cabe inteiramente na área de buffer reservada;

3. Não haverá teste das variações da seletividade do predicado da junção e atente para a melhor opção entre qual tabela (em tamanho) será a interna e a externa no join

5.2 Operações de Seleção

Cada grupo deverá avaliar a execução dos seguintes dois tipos de seleção:

1. Seleção por campo UNIQUE, ou igualdade em campo chave
2. Seleção por faixa de valores em desigualdade (<, <=, >, >=) de campo UNIQUE
3. Seleção por faixa de valores em desigualdade (<, <=, >, >=) de campo NOT UNIQUE

Para cada um dos algoritmos, as seguintes configurações deverão ser testadas:

1. Os campos do predicado da seleção:
    a. Não possuem método de acesso primário ou secundário (não possuem índices)
    b. É o campo da organização primária (campo chave da árvore B+, ou a chave do *hashing*)
    c. Possuem um índice secundário
2. Duas seletividades da operação: baixa (0,1%) e alta (50%)

Recomenda-se verificar o resultado de cada primitiva contra uma implementação de referência (por exemplo, a mesma consulta no SQLite ou no PostgreSQL sobre os mesmos dados).

6. Metodologia de avaliação

Sugere-se que cada experimento varie um fator por vez em torno de uma configuração-base declarada, e que cada medição seja repetida até que a variação entre repetições fique abaixo de 5%, com no mínimo três repetições.

Os livros-texto apresentam fórmulas de custo para cada organização e operação. Elas podem ser usadas para validar as medidas obtidas e constituem um bom parâmetro de validação dos códigos desenvolvidos pela IA.

6.1 Indicadores

1. **Ocupação de memória secundária**: total de blocos do arquivo de dados (incluindo extensão e overflow) e de cada estrutura auxiliar (níveis de índice, diretório, bitmaps, matriz do grid file), antes e depois das cargas de manutenção.
2. **Acessos a blocos** por operação: leituras físicas, gravações físicas, requisições ao buffer pool, acertos e taxa de acerto e, se o grupo modelar, número de seeks. O tempo de relógio pode ser registrado, mas não substitui a contagem.
3. **Custo estimado × medido**: para cada operação, o valor da fórmula correspondente dos livros-texto, calculado com os parâmetros reais, ao lado do valor medido, com explicação das diferenças.

6.2 Fatores a variar

Fator	Níveis sugeridos
Política de substituição de página do buffer	Duas entre: LRU, MRU, FIFO e Clock (segunda

Fator	Níveis sugeridos
pool	chance)
Tamanho das tabelas	pequenas (cabem na área de buffer reservada), médias e grandes (não cabem na área de buffer reservada)
Seletividade (tamanho da resposta)	cerca de 0,1% e 50% dos registros

6.4 Base de dados

Para os testes, sugere-se o esquema de uma universidade, comentado no livro e em sala de aula, contendo entidades abaixo listadas. Porém, o grupo é livre para criar outras tabelas e campos, e alterar o proposto, segundo o que julgar melhor para os seus testes. Em qualquer dos casos, recomenda-se documentar bem o esquema físico utilizado, tanto no relatório do projeto como na apresentação em sala de aula.

- **Aluno** (CPF, DRE, nome, data de nascimento, sexo, endereço, lista de e-mails, curso, coeficiente de rendimento – CR, professor orientador)
- **Curso** (código, nome, escola ou faculdade, carga horária total, professor coordenador, e-mail, endereço da secretaria, lista de telefones etc.)
- **Disciplina** (código, curso, nome, ementa, carga horária, lista de pré-requisitos)
- **Turma** (número, disciplina, semestre e ano, horário, sala, professor)
- **Inscrição_Aluno_Turma** (DRE do aluno, número da turma, frequência, média final)
- **Detalhes_do_Aluno** (CPF, lista de campos de detalhes)

As chaves estrangeiras, os atributos adicionais e os volumes de cada tabela ficam a critério do grupo. O esquema oferece naturalmente tabelas pequenas (Curso), médias (Disciplina, Turma) e grandes (Aluno, Inscrição_Aluno_Turma); campos de alta (CPF, nome) e de baixa cardinalidade (sexo, curso);

Exemplos de consultas podem ser:

1. Seleciona um aluno dado o seu DRE (seleção pontual)
2. Liste os alunos com CR acima de 9 (baixa selectividade)
3. Liste os alunos com CR abaixo de 5 (alta seletividade)
4. Junte Alunos e Inscrições (tabelas grandes)
5. Junte Cursos e Disciplinas (tabelas pequenas)
6. Junte Aluno e Curso (tabela grande e pequena)
7. Junte Aluno e Turma pelo campo Professor (Equijoin em campo NOT UNIQUE)
8. Junte Aluno com Detalhes pelo campo CPF (Equijoin em campo UNIQUE)

7. Uso de IA, entrega e avaliação

O código deve ser desenvolvido no ambiente do grupo com geração assistida por IA. A autoria do grupo é avaliada pela capacidade de encontrar, explicar e corrigir os defeitos do código gerado, e não pelo volume de código.

7.1 Pontos de atenção para o Code Review

Erros que costumam aparecer em código gerado para este tema e que o grupo deve procurar ativamente:

- leitura de blocos fora do Gerente de Buffer Pool (contagem subestimada);
- pin sem unpin correspondente, ou página alterada sem ligar o dirty bit;
- busca binária sobre registros em vez de sobre blocos;
- árvores B+ que não mantém a ocupação mínima, não encadeia as folhas ou não propaga divisões até a raiz;
- hash que ignora a cadeia de overflow na busca ou na remoção;
- junções que usam estruturas da linguagem (listas, dicionários) maiores que a área de buffer reservada;
- deslocamentos e tamanhos de campos variáveis acessados fora da faixa do bloco.

7.2 Produtos a entregar

1. **Repositório** (GitHub Classroom ou GitLab CI) com todo o código, scripts de geração da base e de execução da bancada de teste, e os arquivos de resultados.
2. **Relatório técnico** com:
    a. todas as opções adotadas no projeto: tamanhos de bloco, blocagem, descrição da base, dos campos e das chaves, estruturas dos arquivos e dos cabeçalhos, campos de ordenação, funções de hash, ordem das árvores, configurações da gerência do buffer pool e demais escolhas do grupo.
    b. **Tabelas e gráficos** com os indicadores medidos ao lado dos custos estimados.
    c. **Conclusão:** *seção de análise dos resultados, comparando os resultados de todos os testes e a **lista de recomendações** de configuração física do SBD conforme a carga de trabalho, contendo recomendação sobre qual organização primária, quais índices e qual configuração da gerência do buffer pool adotar em quais casos.*
3. **Relatório de IA**, contendo:
    a. **Log dos prompts** utilizados e o código bruto gerado pela IA.
    b. **Code Review** do código da IA, identificando onde foi gerado código ineficiente ou errado, falhas de eficiência, complexidade de algoritmo, vazamentos de memória, acessos fora da faixa etc.;
    c. **bateria de testes** efetuados em busca de problemas, incluindo testes de estresse.
    d. **Análise Crítica e Post-Mortem de IA**. principal critério de avaliação da autoria, contendo:
    a. Arquivo `POST_MORTEM.md` na raiz do repositório
    b. Code Review crítico: Os prompts-chave e os fragmentos de código brutos devolvidos pela IA, antes de qualquer modificação manual;
    c. Justificativa de refatoração: as alterações feitas para atender aos requisitos de desempenho, correção e estresse;
    d. Evidência de testes: relatório dos testes de estresse mostrando que o código refatorado supera o código ingênuo gerado inicialmente.

7.3 Postagem dos resultados

- Os relatórios em PDF devem conter **folha de rosto** com os nomes completos e o DRE de todos os integrantes.
- Os produtos são postados no AVA@UFRJ por **apenas um** integrante do grupo; os demais postam apenas a folha de rosto em PDF. O aluno que não postar nada, mesmo estando no grupo, terá a nota reduzida, podendo ser zero.
- Cada grupo fará uma **apresentação oral** do trabalho em sala de aula, sendo necessário também a postagem dos slides como resultado no AVA@UFRJ
- Em caso de indisponibilidade do AVA@UFRJ no dia do prazo, será considerado entregue o trabalho (PDF e slides) enviado dentro do prazo para milton@matematica.ufrj.br.